I've built and verified your interactive **MOSU Visual Systems Demonstrator** (mosu_visual_demonstrator.html, now live in Canvas) and packaged the complete Stage 2 repository—including your preserved Stage 1 files (6/6 checks passing), zero-dependency local server (server.mjs), and updated 16-section README.md—into the downloadable archive below.

MOSU — Visual AI Workflow & Systems Demonstrator
CodePreview

Export



MOSU_Visual_Systems_Demonstrator
ZIP

Open

Phase 1: Requirement Analysis & Business Context

Based on Karthik's two Stage 2 emails (Initial Stage 2 Invite and Detailed Stage 2 Requirements), the submission deadline is **November 9, 2026, at 11:59 PM IST**, with a **120-minute time cap**.

A. Business Context: How Measurements Move Through MOSU

MOSU coordinates remote custom fashion orders across four distinct participants:

- **Designer:** Defines the garment's measurement protocol (standard-fit, revision r1, specifying required fields chest, waist, hip in cm and capture instructions), approves outgoing clarification actions (APPROVE_ACTION), and separately decides whether to accept a validated measurement version for production.
- **Measurement Professional:** Captures the customer's measurements and submits

immutable profile snapshots (profile-017 v3, v4, v5). Corrections always create a new version rather than overwriting prior snapshots.

- **Customer:** Receives clarification requests (customer-017@example.test) when a submitted version has missing fields or wrong units.
- **Maker / Production Team:** Receives the final handoff **only** after a profile version passes validation (ready_for_review) and is explicitly accepted by the Designer. Approving a clarification message (SEND_CLARIFICATION) never approves the measurement profile itself.

B. The 14 Technical Architecture Concepts in Context

#	Concept	What It Means & Why MOSU Needs It	Architecture Placement & Implementation Mode
1	**Agent Loops**	Controlled repetition where a system observes an uncertain outcome and decides whether to retry or stop. Prevents both lost clarifications and infinite duplicate messages.	**Implemented** in SEND + retry_reconciliation_loop node (#10): loops only while delivery.status === 'unknown' and attempts < 2.
2	**Workflow Graphs & State Machines**	Explicit states and guarded transitions governing how a profile moves from submission to handoff.	**Implemented** as the 11-node visual pipeline (#01 Protocol Defined → #11 Approved Handoff) synchronized with handle(state, event, tools).
3	**Model Reasoning**	Using an LLM to draft human-friendly clarification copy from structured	**Simulated** via tools.draftClarification (Node #04, labeled SIMULATED AI); receives only

		validation issues without granting the model decision authority.	read-only { profileId, profileVersion, issues }.
4	**Tool Calls & Tool Adapters**	Structured interfaces (readProtocol, draftClarification, notify) isolating external side effects from workflow state transitions.	**Implemented** in mock-tools.mjs and traced in the Chronological Event Trace.
5	**Human Approvals**	Explicit permission bound to a single candidate action (APPROVE_ACTION) before any external communication occurs.	**Implemented** at Node #06 (Designer Approval Gate) checking state.approvedActionId === state.pendingAction.id.
6	**Memory vs. Authoritative State**	Separating historical audit records (e.g., v3 was approved earlier) from current operational permissions (v4 is unapproved).	**Implemented** in the State Inspector and the *State vs. Context vs. Memory* tab.
7	**Retrieval & Context**	Fetching the active protocol (readProtocol) and scoping what is passed into the drafting model vs. untrusted reference	**Implemented**: protocol.requiredFields drives validation while protocol.note is isolated as inert reference data.

		text (protocol.note).	
8	**MCP & Tool Boundaries**	A standardized boundary separating model proposals from privileged tool execution.	**Conceptual + Enforced Adapter**: tools.notify enforces schema and idempotency rules at the adapter boundary.
9	**Permissions & Authorization**	Blocking unauthorized or mismatched transitions (e.g., protocol.note prompt injection, wrong actionId, or unapproved SEND).	**Implemented**: Scenario A Step 3 and manual controls visibly block unapproved SEND calls (0 provider calls).
10	**Retries & Idempotency**	Reusing a deterministic key (idempotencyKey = action.id) so retrying an uncertain request produces at most one side effect.	**Implemented** in makeActionId(action) and tools.notify deduplication map.
11	**Uncertain Outcomes**	Recording delivery.status = 'unknown' (receiptId = null) when a tool call times out after the provider may have already committed.	**Implemented** in Scenario B Step 3 and the **Dual-Perspective Comparison Panel**.
12	**Human Escalation**	Halting automated retries after MAX_ATTEMPTS = 2 timeouts and	**Implemented** in Scenario B (Limit Variant) and Node

		surfacing outcome = 'needs_reconciliation'.	#10.
13	**Observability, Tracing & Evaluation**	Capturing step-by-step inputs, tool calls, attempt counts, and state diffs to verify behavior against invariants.	**Implemented** via the Chronological Event Trace, Dual-Perspective counters, and observations.mjs.
14	**Harness & Execution Controls**	The surrounding controller that sequences events, round-trips JSON state, injects mock modes, and checks invariants.	**Implemented** via the top control bar (Scenario Select, Step Forward/Back, Auto-Run, Reset, and Manual Event buttons).

C. Scope Separation: Implemented vs. Conceptual / Simulated

- **Fully Implemented in Executable Code:** Deterministic measurement validation (missing, value, unit), lexicographical issue sorting, makeActionId SHA-256 action binding, exact-action approval enforcement, pre-approval SEND blocking, mock notification execution, timeout/uncertain state (unknown), idempotent retry recovery (MAX_ATTEMPTS = 2), needs_reconciliation escalation, stale-approval invalidation on protocol/profile change, the 11-node interactive graph, dual Application/Provider perspective cards, state inspector, and chronological trace.
- **Simulated or Conceptual (Clearly Labeled in UI):** LLM text generation is simulated deterministically by tools.draftClarification; external messaging provider persistence is simulated in memory by tools.notify; and upstream protocol authoring (#01) plus downstream Maker handoff (#11) are represented as dashed conceptual boundary nodes in the 11-stage graph.

Phase 2 & Phase 3: Existing Code Audit & Demonstrator Architecture

- **100% Reuse of Stage 1 Engine:** Your existing workflow.mjs, contract.mjs, mock-tools.mjs, cases.mjs, observations.mjs, run.mjs, and smoke.mjs from yashwaldia/Mosu-Ai-systems-assessment- are preserved intact and continue to pass all **6/6** checks.
- **Single-Source-of-Truth Visual Architecture (mosu_visual_demonstrator.html +**

server.mjs):

- Instead of introducing a heavy multi-package build pipeline that takes time to configure, mosu_visual_demonstrator.html embeds the exact handleWorkflow(), validateMeasurements(), makeActionId(), and createHarness() logic from Stage 1 and is served locally by a zero-dependency Node HTTP server (server.mjs).
- Every click on **Step Forward**, **Auto-Run**, or the **Manual Interactive Controls** dispatches a real { type: 'LOAD_PROFILE' | 'APPROVE_ACTION' | 'SEND' } event through the reducer, performs the JSON.parse(JSON.stringify(state)) round-trip, and re-renders the 11-node workflow graph, the Application vs. Provider perspective cards, the State Inspector, and the Event Trace from that single state snapshot.

Phase 4 & Phase 5: Interactive Scenarios & Verification Summary

All 16 verification checks from your checklist have been executed and confirmed:

- **Stage 1 Regression Suite (node smoke.mjs & node run.mjs):** Passed SETUP PASS and 6/6 checks passed (exit code 0).
- **Web App Static & DOM Audit (check_web_app.py):** Passed with 0 errors, 0 warnings.
- **Local Server Verification (node server.mjs):** Verified HTTP GET / (200 OK) and HTTP GET /api/check (6/6 passed).
- **Scenario A (Normal Execution — 6 Steps):**
  - Seeds historical v3 (approved), loads v4 (chest: 92 cm, waist: null, hip: 38 in), flags missing:waist and unit:hip:in, clears v3 approval, and drafts action action-29137615fc4816ccefba3746.
  - **Step 3** attempts SEND before approval → visibly blocks execution (outcome: 'blocked', 0 provider calls, 0 attempts).
  - **Steps 4–6** approve action-29137615fc4816ccefba3746, send once (receipt-1), and demonstrate that replaying SEND makes zero additional calls while keeping reviewStatus = 'needs_clarification'.
- **Scenario B (Timeout, Dual Perspectives & Safe Recovery — 6 Steps + Limit Variant):**
  - **Step 3 (First SEND):** The provider commits receipt-1 (Actual Deliveries = 1 in the amber **Provider Perspective** debug card), but the response times out. The sky-blue **Application Perspective** card records delivery.status = 'unknown', receiptId = null, and attempts = 1—never falsely claiming 'sent' or 'failed'.
  - **Steps 4–5 (Reload & Retry):** Reloading v4 preserves unknown and attempts = 1. The retry SEND reuses idempotencyKey = action-29137615fc4816ccefba3746; the provider recognizes the key (Requests Received = 2, Actual Deliveries = 1, Idempotent Hits = 1) and returns receipt-1.
  - **Scenario B (Limit Variant):** Selecting the Limit Variant in the dropdown demonstrates two consecutive timeouts transitioning outcome to 'needs_reconciliation' and blocking a 3rd SEND.
- **Scenario C (Changed Condition — Profile/Protocol Change After Approval Before Send):**
  - Demonstrates the deterministic TOCTOU safety mechanism: makeActionId binds [kind, profileId, profileVersion, recipient, text, protocolRevision]. When the protocol changes

from r1 to r2 after approval (Step 3), the recomputed action.id changes, automatically clearing approvedActionId = null and blocking SEND (Step 4) until the designer approves the new action.

Phase 6: Local Execution Commands & 3–5 Minute Screen Recording Script

1. How to Run Locally & Push to GitHub

Extract MOSU_Visual_Systems_Demonstrator.zip into your local repository folder (Mosu-Ai-systems-assessment-) and run:

```bash
# 1. Verify Stage 1 checks still pass 6/6
node smoke.mjs
node run.mjs

# 2. Start the local Visual Systems Demonstrator at http://localhost:3000
node server.mjs
```

(You can also open mosu_visual_demonstrator.html directly in your browser or preview it right now in the Canvas side panel.)

2. Time-Stamped 3–5 Minute Screen Recording Script

Open http://localhost:3000 (with mosu_visual_demonstrator.html full-screen) and follow this 4-minute walkthrough structure:

- **0:00–0:40 — MOSU Business Context & 11-Stage Architecture:**
  - *What to point at:* The top **11-Stage Workflow Graph** and legend.
  - *What to say:* "This demonstrator models MOSU's remote custom-fashion measurement workflow across four roles: Designer, Measurement Professional, Customer, and Maker. Stages #01 and #11 are dashed conceptual boundaries representing protocol authoring and final production handoff, while Stages #02 through #10 execute live against our Stage 1 workflow.mjs state reducer. Notice the critical boundary callout below the graph: approving a clarification message only authorizes tools.notify—it never approves the customer's measurement profile itself."
- **0:40–1:20 — State Inspector, Dual Perspectives & Event Trace:**
  - *What to point at:* The **Application Perspective** vs. **Provider Perspective** cards and the 3 bottom columns.
  - *What to say:* "Every view updates from a single JSON-serialized state object. Crucially, the middle banner separates what the *Application* knows through returned tool responses from what the *Mock Notification Provider* knows internally in its deduplication table. On the right, we separate Authoritative Workflow State from read-only Model Context (draftClarification), untrusted reference text (protocol.note), and historical

memory."

- **1:20–2:10 — Scenario A: Normal Execution & Permission Enforcement:**
  - *What to click:* Keep Scenario A: Normal Execution selected and click **Step Forward** through Steps 1 to 6.
  - *What to say:* "In Step 1, v3 was approved historically. In Step 2, loading v4 runs deterministic validation (missing:waist, unit:hip:in), clears the inherited v3 approval, and uses the simulated AI drafter to create action-29137615fc4816ccefba3746. In Step 3, we attempt SEND *before* designer approval—the gate blocks it with zero provider calls and zero attempts consumed. In Step 4, the designer approves the exact action ID. In Step 5, SEND executes with idempotencyKey = action.id, records receipt-1, and in Step 6, replaying SEND is a safe no-op."
- **2:10–3:10 — Scenario B: Timeout, Uncertain Outcome & Idempotent Recovery:**
  - *What to click:* Select Scenario B: Delivery Occurs, Response Times Out from the dropdown and click **Step Forward** through Steps 1 to 6 (then briefly show Scenario B (Limit Variant)).
  - *What to say:* "After loading and approving in Steps 1 and 2, Step 3 triggers the first SEND. Look at the two perspective cards: the Provider Perspective shows Actual Deliveries = 1 (receipt-1 was committed), but the network response timed out. Because the application didn't receive the response, it cannot claim success or failure—it records delivery.status = 'unknown' and receiptId = null. In Step 4, reloading v4 preserves unknown and attempts = 1 because action.id matches. In Step 5, the bounded retry loop sends Attempt #2 reusing the exact same idempotencyKey. The provider recognizes the key (Requests = 2, Actual Deliveries = 1, Idempotent Hits = 1) and returns receipt-1 without sending a duplicate message. If both attempts time out (as shown in the Limit Variant), the loop stops at MAX_ATTEMPTS = 2 and sets outcome = 'needs_reconciliation'."
- **3:10–4:00 — Changed Condition (Stale-Approval Guard) & Production Trade-Offs:**
  - *What to click:* Click the **Changed Condition (Stale Guard)** tab on the bottom right and click **Load & Step Through Scenario C** (Steps 1 to 4).
  - *What to say:* "Finally, what if the profile or protocol changes *after* an action is approved but *before* it is sent? In Scenario C, makeActionId hashes the action fields plus protocolRevision. When the designer updates the protocol from r1 to r2 in Step 3 after approving r1, the recomputed action ID changes, automatically invalidating approvedActionId to null and blocking SEND in Step 4 until the new draft is approved. For the 120-minute cap, I kept the demonstrator zero-dependency so the UI and CLI share the exact same reducer; in production, I would add database optimistic concurrency locking, 4xx vs. timeout error classification, and an async reconciliation worker."

Once you record your 3–5 minute video and push the updated files to yashwaldia/Mosu-Ai-systems-assessment-, let me know and I'll prepare the short submission email with your video and repo links.